

Съгласуване и RLHF

Лекция 8

1.3В срещу 175В

Закачката

InstructGPT — един изненадващ резултат

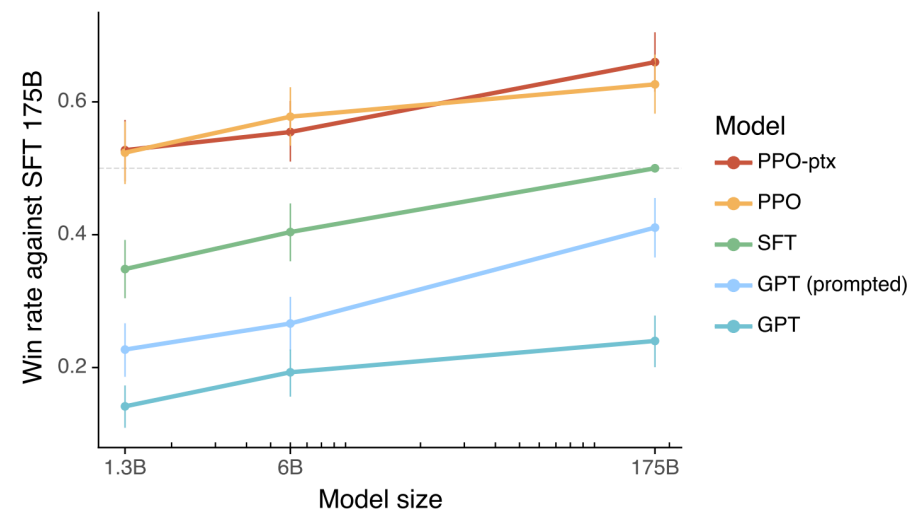
Ouyang et al. (2022) показват нещо странно:

1.3B RLHF модел е предпочитан от хора пред
175B GPT-3 базов модел

По-малкият модел **не може** да знае повече — има
100x по-малко параметри.

Въпросът:

Какво са направили с него, което го прави *по-полезен*
— без да му добавят знание?



Две хипотези

Пред нас стоят две обяснения:

(А) Инструкционният модел **знае повече** — съгласуването го е научило на нови факти и умения.

(Б) Инструкционният модел **знае същото**, но го **разгръща по различен начин** — съгласуването е оформило кога и как да използва вече наученото.

До края на лекцията ще знаем кое е вярно.

Какво всъщност прави базовият модел

Базовият модел е обучен с една цел: **предвиждане на следващия токен**.

Три начина, по които това се проваля като асистент:

1. **Третира подканите като документи за продължаване** — не като заявки за изпълнение
2. **Няма цел "полезен"** — само вероятност; ако продължението на въпроса е друг въпрос, ще го напише
3. **Няма цел "безвреден"** — ако токсичен текст е вероятен в обучаващите данни, ще го продължи

3H рамката

Съгласуването решава три проблема едновременно:

	Проблем	Решение
Helpful (Полезен)	Продължава текст, не изпълнява заявки	SFT на демонстрации
Honest (Честен)	Халюцинира с увереност	RLHF с предпочитания за точност
Harmless (Безвреден)	Продължава вреден текст	RLHF с безвредни данни

Базовият модел не е нито едно от трите **по дизайн** — обучителната му цел е само вероятност.

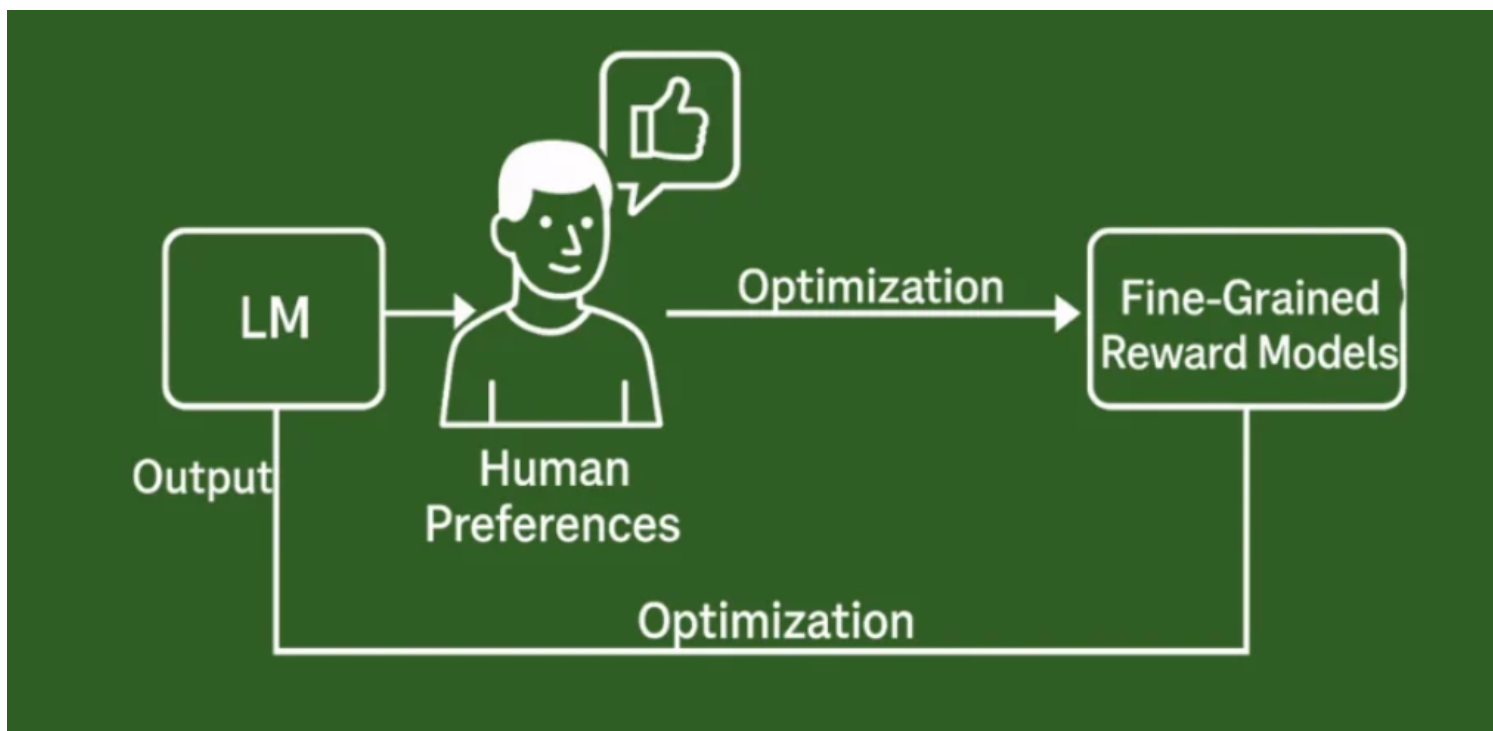
↓ Тетрадка: Демо

Демото продължава в тетрадката

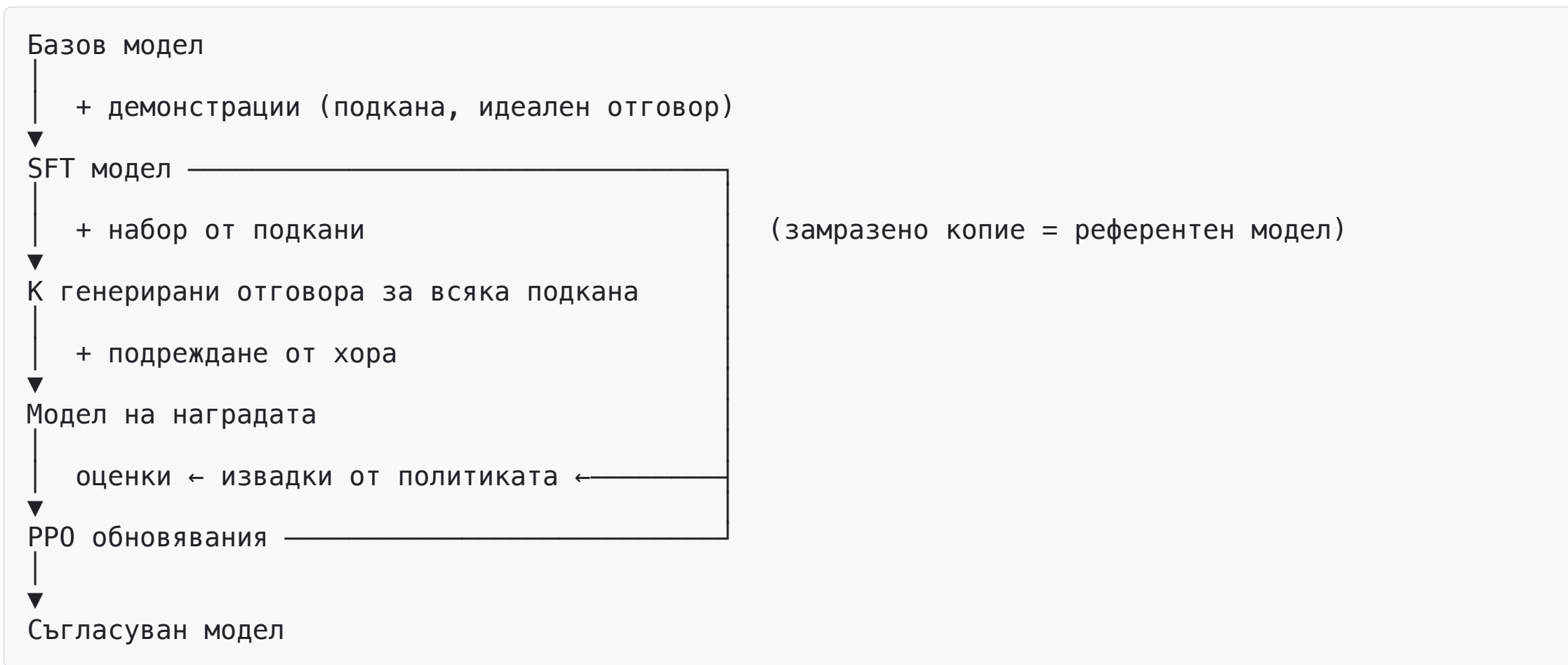
Методи за съгласуване

SFT → Данни за предпочитания → RM → RLHF

RLHF — ОСНОВНИЯТ ЦИКЪЛ



Целият процес — от един поглед



SFT моделът инициализира и RM, и RL политиката. Всичко, което следва, тръгва от тази контролна точка.

SFT — Supervised Fine-Tuning

Същата обучителна цел като при предварителното обучение — **предвиждане на следващия токен**.
Но върху **подбрани** (подкана, идеален_отговор) двойки.

Ключова разлика: загубата се прилага само върху токените на **отговора**, не на подканата.

Подкана: "Кой е авторът на 'Под игото'?" ← без загуба
Отговор: "Авторът на 'Под игото' е Иван Вазов." ← загубата се смята тук

Моделът учи формата на асистент — не нова информация.

SFT данни — какво съдържат

SFT наборите от данни учат модела на **формат на асистент**:

- Обикновени въпроси и инструкции
- Многоходови диалози със системни подкани → поведение при роля
- Трасета на извикване на инструменти
- Структурирани изходи (JSON, схеми)
- Трасета с мисловни вериги — разсъждение стъпка по стъпка
- Откази и калибрирана неувереност

Когато използвате инструкционен модел от L10 нататък — всичко, което виждате (JSON изход, извикване на инструменти, откази) е поставено там от SFT данни. Архитектурата е **идентична** с базовия модел.

Защо SFT само не е достатъчно

SFT учи модела как изглежда **един** добър отговор.

Но не го учи как да **подрежда** отговори или да избягва лоши такива, които не е виждал.

Проблемът: хората са по-добри в преценката ("кой е по-добър?") отколкото в демонстрацията ("покажи перфектен отговор").

Решението: да използваме тази по-лесна задача — **подреждане** вместо писане. Това е ядрото на RLHF.

Генериране на кандидати за сравнение

Как се създават обучаващите данни за RM:

1. Взимаме SFT модела и нов набор от подкани (не от обучаващото множество на SFT)
2. За всяка подкана генерираме **K отговора** ($\text{temperature} > 0$)
Един и същ модел, една и съща подкана, K различни отговора — защото генерирането е стохастично
3. Хора виждат **двойки** и избират по-добрия — не пишат нищо
4. Един избор = един обучаващ пример: (подкана, избран, отхвърлен)

Ключова точка: подреждането е по-лесно от писането. Затова RLHF мащабира отвъд SFT.

Пример: 4 кандидата за една подкана

Подкана: "Обясни какво е машинно самообучение на 10-годишно дете."

#	Отговор (съкратен)	Качество
1	"Машинното самообучение е подобласт на компютърните науки, занимаваща се с алгоритми, които позволяват на системите да се учат от данни..."	Технически, неподходящ за дете
2	"Представи си, че учиш куче на трикове с бисквитки..."	Добра аналогия, ясен
3	"Компютрите могат да се учат! Ако покажеш на компютъра много снимки..."	Добър, но по-дълъг
4	"Е, трудно е да се обясни..."	Неинформативен

Хората подреждат двойки: $2 > 1$, $2 > 4$, $3 > 1$, ...

Модел на наградата — какво представлява

Моделът на наградата е **пълноценен езиков модел** — инициализиран от SFT модела.

Промяна само в последния слой: вместо проекция към vocab_size вероятности → **един скалар**.

Вход: подкана + отговор (конкатенирани)

Изход: едно число — "колко биха харесали хората този отговор"

Не е базиран на правила. Не е класификатор.

Научена функция: (подкана, отговор) → скалар за предпочитание.

За да съди отговор, трябва да разбира въпроса поне толкова добре, колкото отговарящият.

InstructGPT използва 6B RM за обучение на 175B политика.

Обучение на модела на наградата

Данни: предпочитани двойки (подкана, $y_{\text{избран}}$, $y_{\text{отхвърлен}}$)

RM оценява и двете:

$$r_{\text{избран}} = \text{RM}(\text{подкана}, y_{\text{избран}})$$

$$r_{\text{отхвърлен}} = \text{RM}(\text{подкана}, y_{\text{отхвърлен}})$$

Bradley-Terry загуба:

$$\mathcal{L} = -\log \sigma(r_{\text{избран}} - r_{\text{отхвърлен}})$$

Загубата иска: $r_{\text{избран}} > r_{\text{отхвърлен}}$, с голяма разлика.

Важно: **абсолютните стойности нямат значение** — само разликата. RM учи да **подрежда**, не да дава калибрирани числа.

Модел на наградата — ограничения

RM компресира предпочитанията на група хора в една функция. Той наследява техните пристрастия и особености.

Манипулиране на наградата (Reward hacking):

Политиката открива отговори, които максимизират оценката на RM, но са очевидно лоши — многословни, прекалено уклончиви, сервилни. RM е научил корелация, не истината.

Пример: "Разбира се! Чудесен въпрос! Ще се постарая да ви помогна по най-добрия начин. Що се отнася до вашия въпрос..." [продължава без отговор]

Преход: имаме функция за оценяване. Последната стъпка е да я използваме за обучение на политиката. Тук влиза RL.

Трите модела в паметта

По едно и също време в GPU паметта живеят три модела:

Модел	Роля	Обновява се?
Политика	Генерира отговори; именно него обучаваме	Да — PPO
Референтен модел	Замразено копие на SFT модела; измерва отклонението	Не
Модел на наградата	Оценява отговорите	Не

Три големи модела в паметта едновременно. Затова RLHF е скъп.

Една стъпка от PPO цикъла

1. Избираме подкана от набора
2. Политиката генерира отговор y (токен по токен, с температура)
3. RM го оценява: $r = \text{RM}(\text{подкана}, y)$ — едно число
4. При всеки токен измерваме KL дивергенция между политика и референтен модел
5. Ефективна награда:

$$R = r - \beta \cdot \text{KL}(\text{политика} \parallel \text{референтен})$$

6. PPO изчислява обновяване на градиента с ограничена целева функция
7. Обратно към стъпка 1 с нова подкана

Ключова разлика от SFT: **политиката генерира собствените си обучаващи данни** в реално време. Разпределението се измества с всяко обновяване.

KL наказание — защо е ключово

Без KL наказание: политиката открива генерации, максимизиращи оценката на RM — почти винаги безсмислици, манипулиращи RM. Забравя как да говори в гонитба за висока награда.

С KL наказание: политиката може да се отдалечи само дотолкова от SFT модела. Остава върху многообразието от "неща, които SFT моделът би казал".

Референтният модел е каишката. Моделът на наградата е лакомствата. PPO обучава кучето да гони лакомствата, но каишката го спира да се хвърли от скала.

Дълбока връзка с тезата: KL наказанието е математическото твърдение, че знанието от предварителното обучение трябва да бъде запазено. Обучението с подсилване може само да насочи — не да корумпира.

Какво всъщност прави PPO

PPO решава конкретен проблем: как да обновяваме политиката, без да я разрушим с една голяма крачка.

Идеята: измерваме съотношението между новата и старата политика за всеки токен.

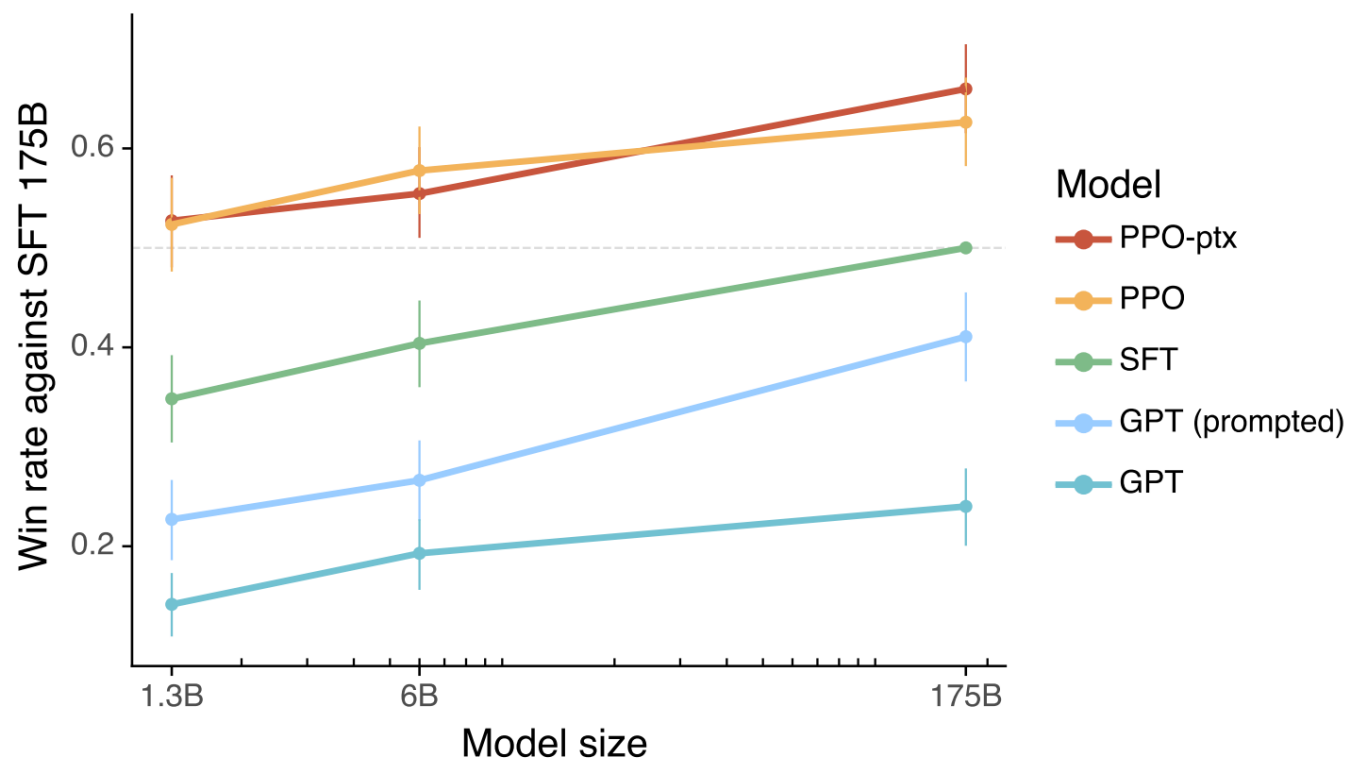
Ако новата политика дава много по-висока (или по-ниска) вероятност за даден токен в сравнение с предишната — спираме да включваме тази стъпка в градиента.

На практика: ако съотношението излезе извън тесен диапазон (около 0.8–1.2), то се "отрязва" — изключва се от изчислението.

Обнови в тази посока — но не прекалено далеч.

Ефектът: обучението е стабилно, но бавно. Нужни са много итерации, три модела в памет едновременно, и чувствителна настройка на хиперпараметри. Точно тези ограничения мотивират DPO.

Резултатът: кръгът се затваря



Сега разбираме защо.

По-малкият модел не знае **повече**. Той е научен да **разгръща** знанието си по-полезно.

Хипотеза (Б) е вярна.

DPO — Direct Preference Optimization

Нужен ли ни е RM като отделен артефакт?

DPO (Rafailov et al. 2023): обединява RM и RL стъпката в **една загуба с учител** върху предпочитани двойки.

- Без RM. Без генериране на кандидати. Без PPO.
- Директно от (подкана, избран, отхвърлен) → обновяване на политиката.

Сега стандарт за повечето модели с отворен код — Llama-3, Mistral, Qwen инструкционни варианти.

Няма да разглеждаме механиката — само идеята и мотивацията.

Разсъждаващи модели — кратко споменаване

Същата PPO/RL машинария — но наградата идва от проверка на отговорите, не от научен RM:

- Математика може да се верифицира
- Код може да се изпълни
- Логически задачи имат верен отговор

Резултатът: модели с дълги вътрешни мисловни вериги преди отговора (o1, DeepSeek-R1).

Не е нова архитектура — RLHF с по-умна награда.

LIMA и преходът към почивката

Zhou et al. (2023): Llama-65B, с фина настройка на **само 1,000 подбрани примера**, без RLHF.

Резултат: конкурентен с RLHF-обучени модели. Още едно потвърждение на хипотеза (Б) — знанието е от предварителното обучение.

Superficial Alignment Hypothesis:

"A model's knowledge and capabilities are learnt almost entirely during pretraining, while alignment teaches it which subdistribution of formats should be used when interacting with users."

Когато разгледаме реалните набори от данни — питайте се: изглежда ли като предаване на ново знание, или като оформяне на съществуващо поведение?

Данни, безопасност и следващото

Асиметрията на усилието

Количествата: предварително обучение срещу съгласуване

	Предварително обучение	Съгласуване
LLaMA 2 данни	~2 трилиона токена	~100K предпочитани двойки
LLaMA 2 изчислителни ресурси	~1.7M GPU-часа	Дни на малък клъстер
InstructGPT данни	~300B токена	13K SFT демо + 33K сравнения
LIMA	~1.4T токена предв. обучение	1,000 примера
Кой го е направил	Анонимно събиране от уеб	~40 наети изпълнители
Продължителност	Месеци	Седмици

Предварителното обучение е там, където отиват парите, изчислителните ресурси, данните и въглеродът. Съгласуването е там, откъдето идва продуктът.

Тънкият слой, без който нищо не работи

Без съгласуването никой не би използвал големите езикови модели.

ChatGPT е съгласуване. GPT-3 съществуваше две години преди ChatGPT — и почти никой извън изследователските екипи не го използваше.

Предварителното обучение създава айсберга. Съгласуването решава накъде да се обърне.

↓ Тетрадка

Реални набори от данни за съгласуване

Jailbreaks като доказателство

Ако съгласуването е тънък поведенчески слой върху дълбоко предварително обучено знание — jailbreaks са **базовият модел, просмукващ се навън**.

- Чрез внимателно конструирани подкани потребители могат да накарат съгласувани модели да произведат съдържание, за което са обучени да откажат
- Няколкостотин примера с фина настройка **могат да отменят RLHF изцяло**

Съгласуването е реално, но **крехко**. Това не е грешка — е математически неизбежно при KL-ограничено обучение.

Заключение

Всяка способност, върху която ще надграждаме от Л10 до Л12, зависи от модел, преминал през пълния процес на съгласуване.

Не защото съгласуването е **научило** тези способности — а защото ги е **отключило** от предварителното обучение.

Когато използвате инструкционен модел от сега нататък, помнете: предварителното обучение върши по-голямата работа. Съгласуването само казва в коя посока да се обърне.